

To centrally stop or intercept browser forward and backward navigation in a Next.js application, you can use the `popstate` event listener or the `beforePopState` method for the Pages Router. [↗](#)

While modern browsers do not allow you to "disable" these buttons for security and usability reasons, you can effectively "trap" the user on the current page or show a confirmation prompt. [👉 Stack Overflow +3](#)

Method 1: Global Interception (Pages Router)

For apps using the Pages Router, you can use `router.beforePopState` in your `_app.js` file to centrally control navigation when the back or forward buttons are pressed. [↗](#)

javascript

```
// pages/_app.js
import { useEffect } from 'react';
import { useRouter } from 'next/router';

export default function MyApp({ Component, pageProps }) {
  const router = useRouter();

  useEffect(() => {
    // This callback runs on every back/forward action
    router.beforePopState(({ url, as, options }) => {
      const confirmLeave = window.confirm("You have unsaved changes. Stay on this page?");

      if (!confirmLeave) {
        // Prevent the URL from changing in the address bar
        window.history.pushState(null, '', router.asPath);
        return false; // Blocks the Next.js router from handling the change
      }
      return true; // Allow navigation
    });

    return () => {
      router.beforePopState(() => true);
    };
  }, [router]);

  return <Component {...pageProps} />;
}
```

Use code with caution.

Method 2: Global `popstate` Listener (App Router)

Since the App Router does not have a direct `beforePopState` equivalent yet, you can use a root layout or a client-side provider to listen for the `popstate` event. 🤖 Reddit · r/nextjs +2

javascript

```
// components/NavigationGuard.tsx (Client Component)
'use client';

import { useEffect } from 'react';
import { usePathname } from 'next/navigation';

export default function NavigationGuard() {
  const pathname = usePathname();

  useEffect(() => {
    const handlePopState = (event) => {
      // Re-push the current state to the history stack to "undo" the back/forward action
      window.history.pushState(null, '', window.location.href);
      alert("Navigation via browser buttons is disabled for this section.");
    };

    window.addEventListener('popstate', handlePopState);

    // Initial push to ensure there's a state to stay on
    window.history.pushState(null, '', window.location.href);

    return () => {
      window.removeEventListener('popstate', handlePopState);
    };
  }, [pathname]);

  return null;
}
```

Use code with caution.

Key Techniques Summary

- `window.history.pushState` : Manually adding a new entry to the history stack every time the user tries to leave effectively traps them on the current page.
- `window.onbeforeunload` : Use this to catch page refreshes or tab closures, though it only triggers a standard browser dialog rather than a custom UI.
- `window.history.forward()` : A legacy "trick" that immediately forces the browser forward if the back button is clicked, preventing the user from staying on the previous page. 🗨️ Reddit · r/nextjs +4

Warning: Completely disabling navigation buttons is generally considered a poor user experience and can lead to users feeling "trapped" on your site. It is recommended to only use these methods for critical workflows, such as preventing data loss in a complex form.